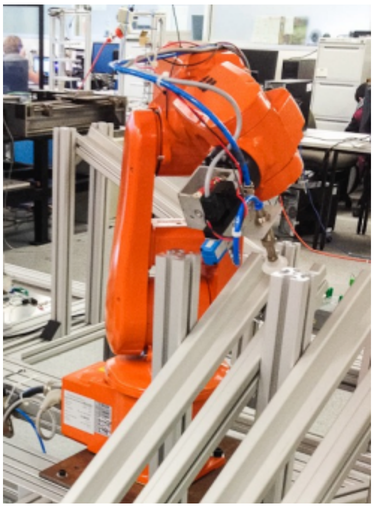
UNIVERSITY
OF WOLLONGONG
AUSTRALIA

LAB REPORT

Subject: MECH470/970

Robotic Sorting System



Week 8

Group 1
Team A

Student Name	Student Number
Harrison Cooke	7138295
Samuel Tancred	7448569
Yasan Gunathilaka	7549246

24th September 2025

1. Objective

The objective of this MECH470 lab is to develop a PLC ladder logic program to control a block sorting robot. The task requires the automated sorting of two blocks, including painted and unpainted. The robotic manipulator has pre-programmed movement instructions and operates at 24V. The PLC program is used to realise the logical component of the sorting task. The digital inputs include 4 position sensors that are triggered as the robot arrives at a location, and one input signal that alerts when the robot is in motion. Additionally, an analogue shade sensor is positioned under the block stack and, with the aid of an illumination source, can infer if the block is painted or unpainted. The digital outputs for the system include the illumination source, the vacuum used to pick up the block and four distinct outputs to instruct the robot to move to the desired location.

2. Lab Tasks and Design Justification

Determine the voltage output for unpainted and painted

The shade sensor is mounted underneath the block slide and sends an analogue output to the PLC. To achieve an accurate and robust reading, a neighbouring light is illuminated while the sensor reads the analogue signal. If the block is painted or unpainted, the shade sensor will have a different signal value, which allows the PLC program to infer the difference between the two. By running a preliminary program illuminating the light and reading the analogue signal, we determined the sensor value for the painted blocks to be in the range 3500 to 7000 and the unpainted block to be between 1000 and 3500. This was simple to implement into our PLC logic using an IN_RANGE block; if the value is within the two determined values, the signal is active. The sensor light is illuminated once the robot reaches the pick-up location.

Create a control program to control the robot's motion

To sequence the robot's motion, we started the program by sending the robot to the 'move2pickup' location if the robot was not already in motion to another location. At the pickup location, the light and shade sensor is triggered to read the block type. Depending on the value of the shade sensor, the robot will be sent to 'move2painted' or 'move2unpainted'. The 'moving' signal of the robot is used to prevent additional movement signals from being sent to the robot while it is already in motion. When the PLC receives a 'not_moving' signal, and hence the robot has arrived at location 'painted' or 'unpainted', the robot is sent back to the pickup location, and the sequence is repeated.

Control the robot's logic to sort the blocks

To complete the main task, the PLC ladder logic pieces previously stated are combined to operate the robot to complete the sorting operation. The shade sensor, vacuum and illumination source are all triggered once the robot reaches the pickup location (loc_pickup) and is sealed in with the vacuum output. The vacuum remains on until the variable 'drop_block' stops the vacuum, and the block is dropped. The drop block variable is activated when the robot reaches loc_painted or loc_unpainted. This was found to be a robust program, as the block was only dropped once the robot arrived at either of the target locations. We encountered an issue with the sensor data reading values of both blocks and sending two signals to the robot's movement queue. To resolve this, we used TON timers in line with the IN_RANGE blocks to ensure that only values in range that were read for 3s consecutively could cause the robot to move to the target location. Additionally, we altered the initial PLC logic to only record the shade sensor value when the robot is at the pickup location. To achieve this, the MOVE block was moved in line with the location pickup switch and is no longer sealed in with the vacuum output.

Once the robot had reached the target location and dropped the block, we wanted to repeat the sequence. To achieve this, a ladder rung was added that, when the robot had no instruction in the queue and was not moving, the robot would be sent to the pickup location, and the sequence would be repeated.

Additional tasks

Two out of three additional tasks were completed within the time constraints. We successfully implemented an alert if the robot wasn't functioning correctly, in addition to the alert when there are no blocks in the slide. With more time, the group could easily implement an alert when the shade sensor or illumination system had failed.

- **Robot malfunction alert system**
  Our fault scenario was designed such that if the robot successfully moved to the pickup location and the vacuum did not turn on, an error would be thrown to one of the system LEDs. This is simple to implement and only requires one ladder logic rung.

- **Determine no blocks in the slide**
  To determine if there were no blocks in the slide, we took a reading of the ambient shade sensor with no blocks and determined it to be around 2000. If the shade sensor detected that signal for more than 2 seconds at the pickup location, the alert to the LEDs would be triggered.

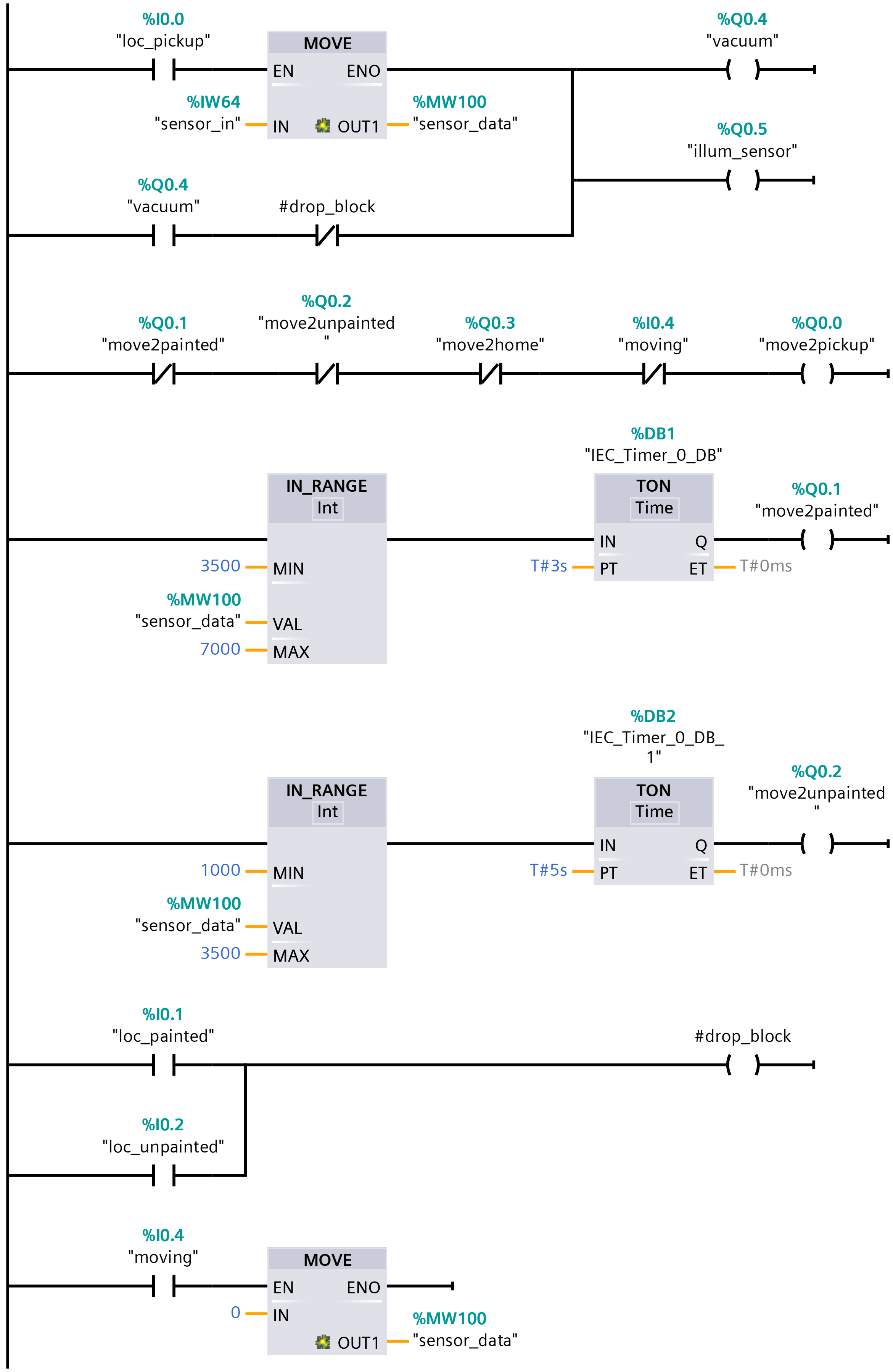
3. Reflection & Conclusion

Design limitations and challenges

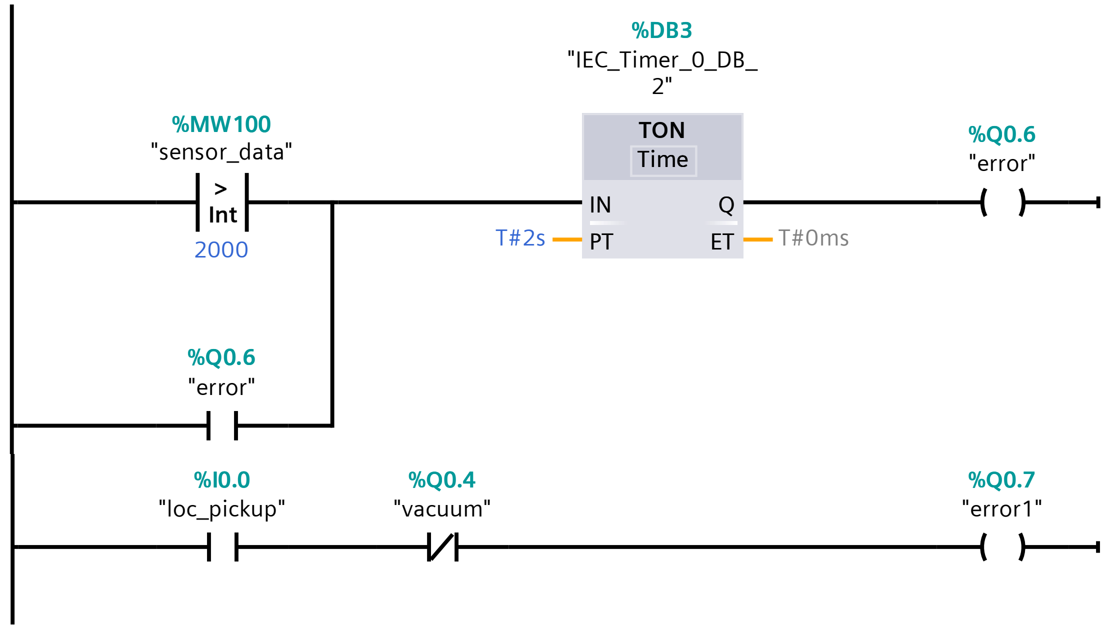
The group completed two of the extra activities in addition to the main task. With more time, the last activity would have been easily implemented; however, due to time restrictions, the task was not completed. One of the most time-consuming issues we encountered was that after picking up the block, the robot would move to both locations before dropping the block. To solve this, we implemented TON timers to ensure that the correct data would be sent to the PLC. Most of the time was spent troubleshooting rather than redesigning the ladder logic. This experience aided our problem-solving ability in correcting noisy data and improving the system's robustness. Another important lesson was the use of conditional logic to prevent conflicting robot instructions. By implementing the robot's 'movement' signal, we ensured that new commands were not issued until the robot had stopped moving.

This issue is difficult to prepare for and requires a refined troubleshooting process to resolve. Despite adequate preparation, some challenges are faced in the implementation process. The robot sorting lab developed our group's troubleshooting process. We used video recordings on our phones of the PLC ladder logic in real-time to aid the group in finding a solution. As the robot was to be shared by two groups, having the recording meant that we could watch back the entire system logic as it runs and develop solutions without the need to re-run the program with the robot.

4. Appendix



%I0.0
"loc_pickup"
MOVE
EN
ENO
%IW64
"sensor_in"
IN
OUT1
%MW100
"sensor_data"
%Q0.4
"vacuum"
%Q0.5
"illum_sensor"
%Q0.4
"vacuum"
#drop_block
%Q0.1
"move2painted"
%Q0.2
"move2unpainted"
%Q0.3
"move2home"
%I0.4
"moving"
%Q0.0
"move2pickup"
IN_RANGE
Int
3500
MIN
%MW100
"sensor_data"
VAL
7000
MAX
%DB1
"IEC_Timer_0_DB"
TON
Time
IN
Q
T#3s
PT
ET
T#0ms
%Q0.1
"move2painted"
IN_RANGE
Int
1000
MIN
%MW100
"sensor_data"
VAL
3500
MAX
%DB2
"IEC_Timer_0_DB_1"
TON
Time
IN
Q
T#5s
PT
ET
T#0ms
%Q0.2
"move2unpainted"
%I0.1
"loc_painted"
#drop_block
%I0.2
"loc_unpainted"
%I0.4
"moving"
MOVE
EN
ENO
0
IN
OUT1
%MW100
"sensor_data"


%DB3
"IEC_Timer_0_DB_
2"
%MW100
"sensor_data"
>
Int
2000
TON
Time
IN
Q
PT
ET
T#2s
T#0ms
%Q0.6
"error"
%Q0.6
"error"
%I0.0
"loc_pickup"
%Q0.4
"vacuum"
%Q0.7
"error1"